

Class 7: Machine Learning 1

Kate Ruiz (PID: A17671200)

Background

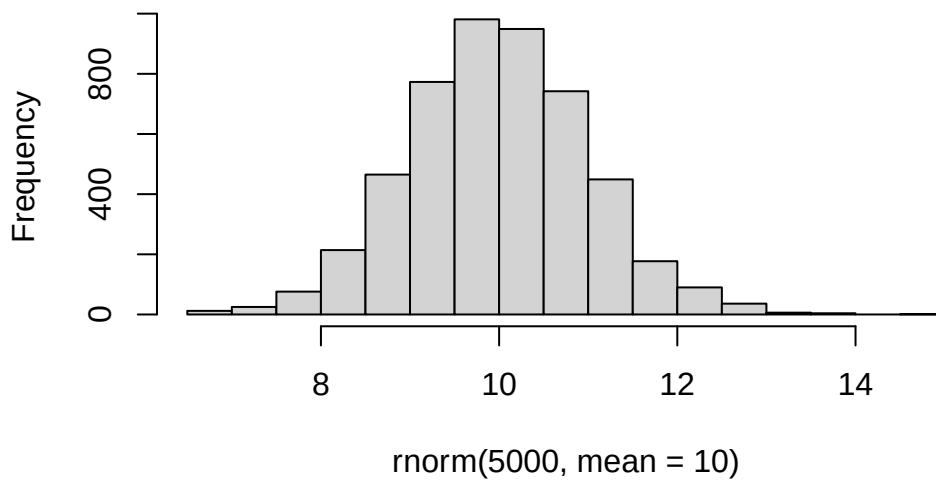
Today we will begin our exploration of some important machine learning methods, namely **clustering** and **dimensionality reduction**.

Let's make up some input data for clustering where we know what the natural "clusters" are.

The function `rnorm()` can be useful here.

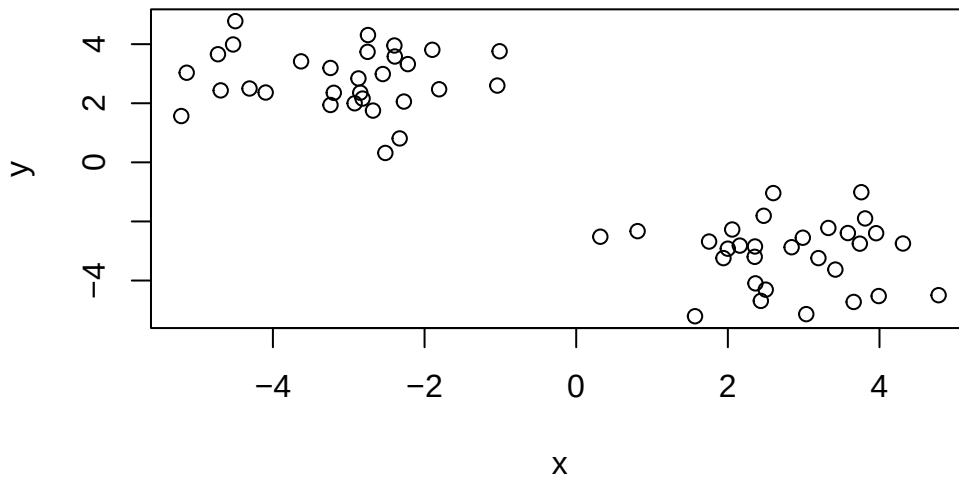
```
hist(rnorm(5000, mean=10))
```

Histogram of `rnorm(5000, mean = 10)`



Q. Generate 30 random numbers at +3

```
tmp <- c(rnorm(30, mean=3),
        rnorm(30, mean=-3))
x <- cbind(x=tmp, y=rev(tmp))
plot(x)
```



K-means clustering

The main function in “base R” for K-means is called `kmeans()`

```
km<-kmeans(x,centers=2)
km
```

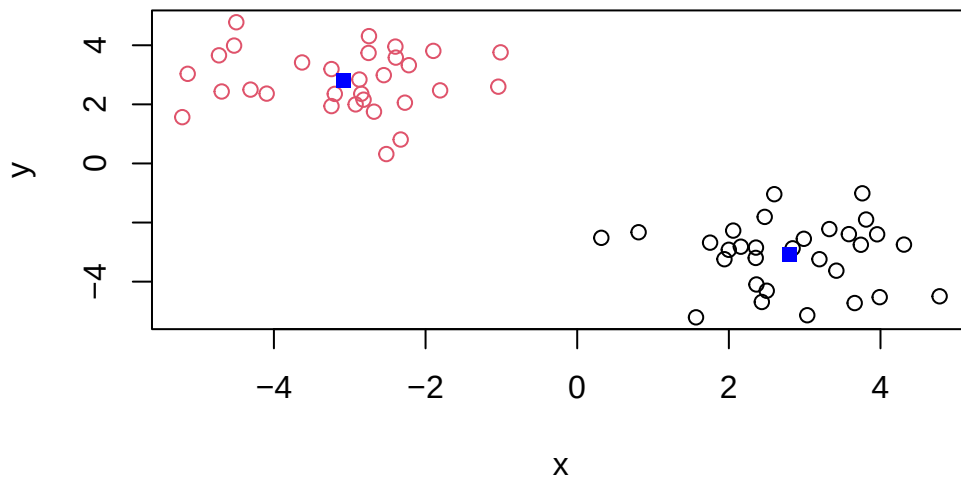
K-means clustering with 2 clusters of sizes 30, 30

Cluster means:

	x	y
1	2.802458	-3.085528
2	-3.085528	2.802458

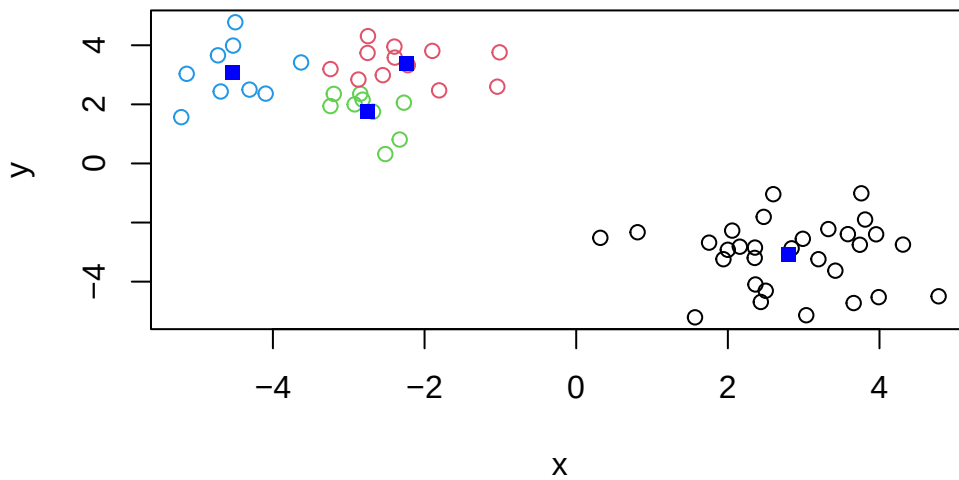
Clustering vector:

```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2
```

Q. Run `kmeans()` again and this time produce 4 clusters (and call your result object `k4`) and make a results figure like above?

```
k4 <- kmeans(x, centers=4)
plot(x, col=k4$cluster)
points(k4$centers, col="blue", pch=15)
```



The metric

```
km$tot.withinss
```

```
[1] 133.2858
```

```
k4$tot.withinss
```

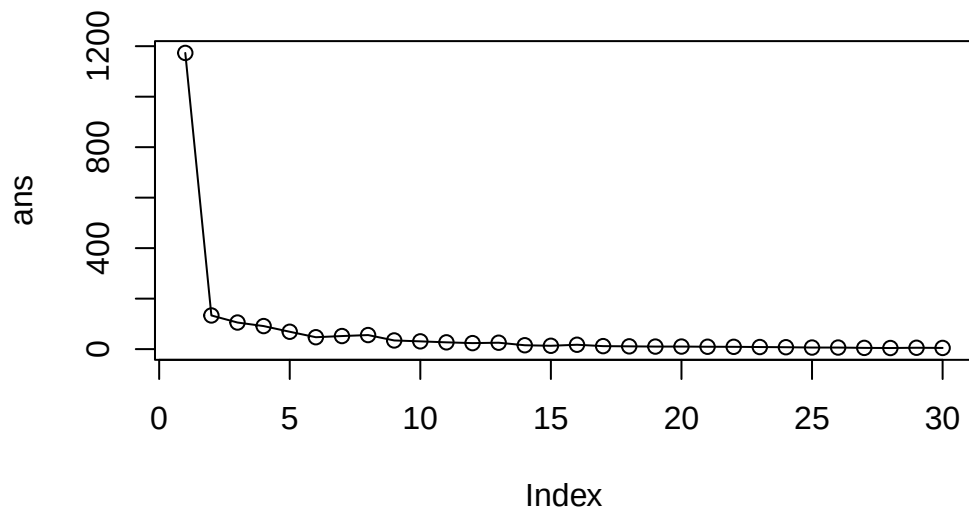
```
[1] 90.21661
```

Q. Let's try different number of K (centers) from 1 to 30 and see what the best result is?

```
ans <- NULL
for(i in 1:30) {
  ans <- c(ans, kmeans(x, centers=i)$tot.withinss)}
ans
```

```
[1] 1173.337240 133.285833 105.270967 91.231268 68.799416 47.147394
[7] 51.801686 55.668997 34.299647 30.755394 26.884849 23.818218
[13] 25.384783 15.334431 13.197944 17.349450 11.859687 10.946731
[19] 10.032048 10.174235 9.386957 9.125443 8.242243 7.553101
[25] 6.192000 6.187150 4.835034 4.299792 5.363894 4.666055
```

```
plot(ans, type="o")
```



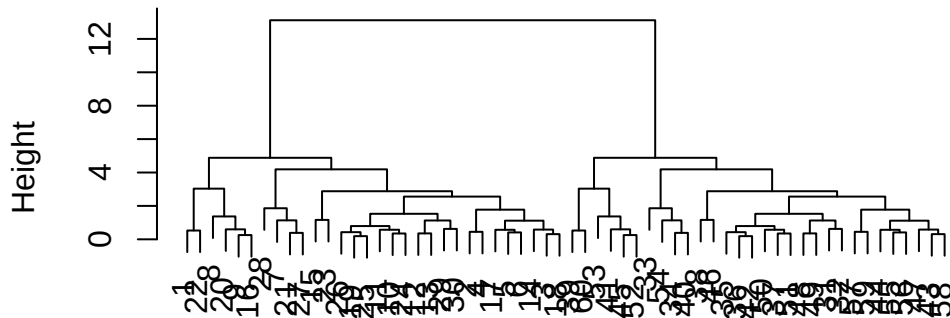
The problem is that it is self-fulfilling. K-means will impose a clustering structure on your data even if it is not there— it will always give you the answer you asked for even if that answer is silly!

Hierarchical Clustering

The main function for Hierarchical Clustering is called `hclust()` Unlike `kmeans()` (which does all the work for you), you can't just pass `hclust()` our raw input data. It needs a “distance matrix” like the one returned from the `dist()` function.

```
d <- dist(x)
hc <- hclust(d)
plot(hc)
```

Cluster Dendrogram



d
hclust (*, "complete")

To extract our cluster membership vector from a `hclust()` result object we have to “cut” our tree at a given height to yield separate “groups”/“branches”.

```
plot(hc)
abline(h=8, col="red", lty=2)
```

d

```
grps <- cutree(hc, h=8)
grps
```

[illegible]

```
grps  1  2
    1 30  0
    2  0 30
```

Import the dataset of food consumption in the UK:

8

	X	England	Wales	Scotland	N.Ireland
1	Cheese	105	103	103	66
2	Carcass_meat	245	227	242	267
3	Other_meat	685	803	750	586
4	Fish	147	160	122	93
5	Fats_and_oils	193	235	184	209
6	Sugars	156	175	147	139
7	Fresh_potatoes	720	874	566	1033
8	Fresh_Veg	253	265	171	143
9	Other_Veg	488	570	418	355
10	Processed_potatoes	198	203	220	187
11	Processed_Veg	360	365	337	334
12	Fresh_fruit	1102	1137	957	674
13	Cereals	1472	1582	1462	1494
14	Beverages	57	73	53	47
15	Soft_drinks	1374	1256	1572	1506
16	Alcoholic_drinks	375	475	458	135
17	Confectionery	54	64	62	41

Q1. How many rows and columns are in your new data frame named x? What R functions could you use to answer this questions?

```
dim(x)
```

```
[1] 17  5
```

```
#rownames(x)
```

```
x[,1]
```

```
[1] "Cheese"           "Carcass_meat "    "Other_meat "
[4] "Fish"             "Fats_and_oils "   "Sugars"
[7] "Fresh_potatoes "  "Fresh_Veg "       "Other_Veg "
[10] "Processed_potatoes " "Processed_Veg "    "Fresh_fruit "
[13] "Cereals "         "Beverages"        "Soft_drinks "
[16] "Alcoholic_drinks " "Confectionery "
```

```
rownames(x)<- x[,1]
```

To remove the first column I can use the minus index trick

```
x <- x[,-1]
x
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139
Fresh_potatoes	720	874	566	1033
Fresh_Veg	253	265	171	143
Other_Veg	488	570	418	355
Processed_potatoes	198	203	220	187
Processed_Veg	360	365	337	334
Fresh_fruit	1102	1137	957	674
Cereals	1472	1582	1462	1494
Beverages	57	73	53	47
Soft_drinks	1374	1256	1572	1506
Alcoholic_drinks	375	475	458	135
Confectionery	54	64	62	41

A better way to do this is to set the row names to the first column by with `read.csv()`

```
x <- read.csv (url,row.names=1)
x
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139
Fresh_potatoes	720	874	566	1033
Fresh_Veg	253	265	171	143
Other_Veg	488	570	418	355
Processed_potatoes	198	203	220	187
Processed_Veg	360	365	337	334
Fresh_fruit	1102	1137	957	674
Cereals	1472	1582	1462	1494

Beverages	57	73	53	47
Soft_drinks	1374	1256	1572	1506
Alcoholic_drinks	375	475	458	135
Confectionery	54	64	62	41

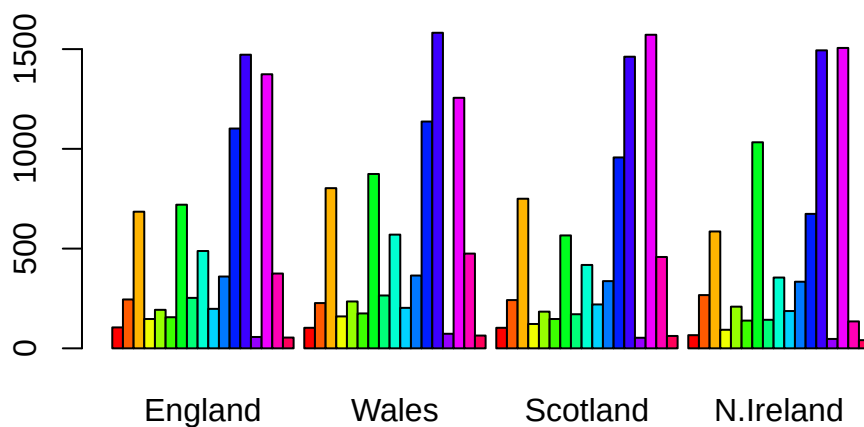
Q2. Which approach to solving the ‘row-names problem’ mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?

The second one. I don’t lose data. I like fixing things at the source.

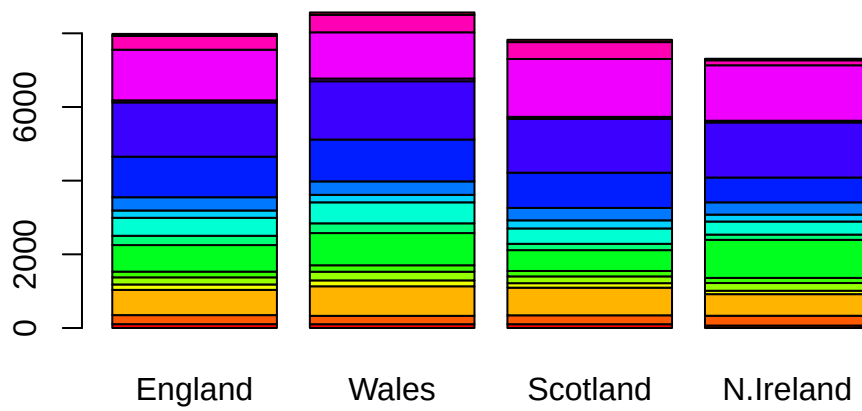
Spotting major differences and trends

is difficult in this wee 17D dataset...

```
barplot(as.matrix(x), beside=T, col=rainbow(nrow(x)))
```

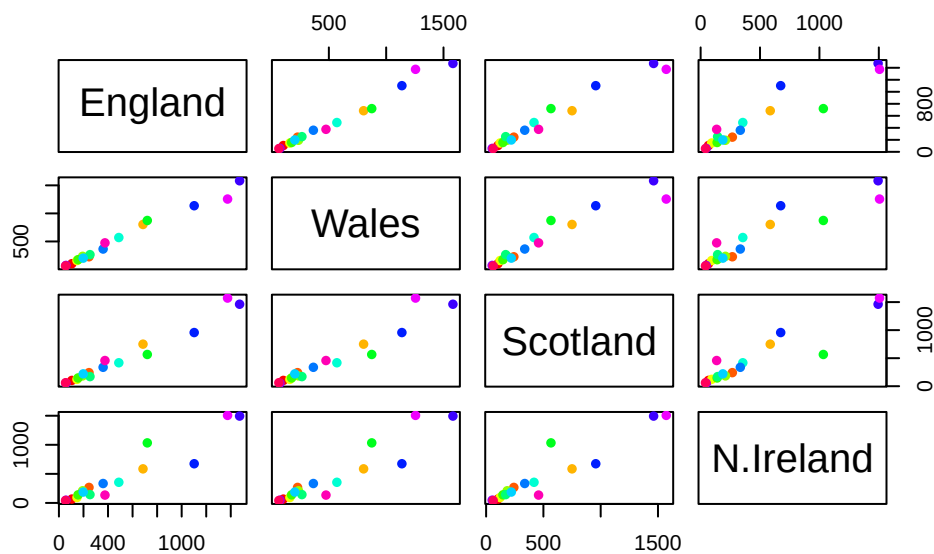


```
barplot(as.matrix(x), beside=F, col=rainbow(nrow(x)))
```

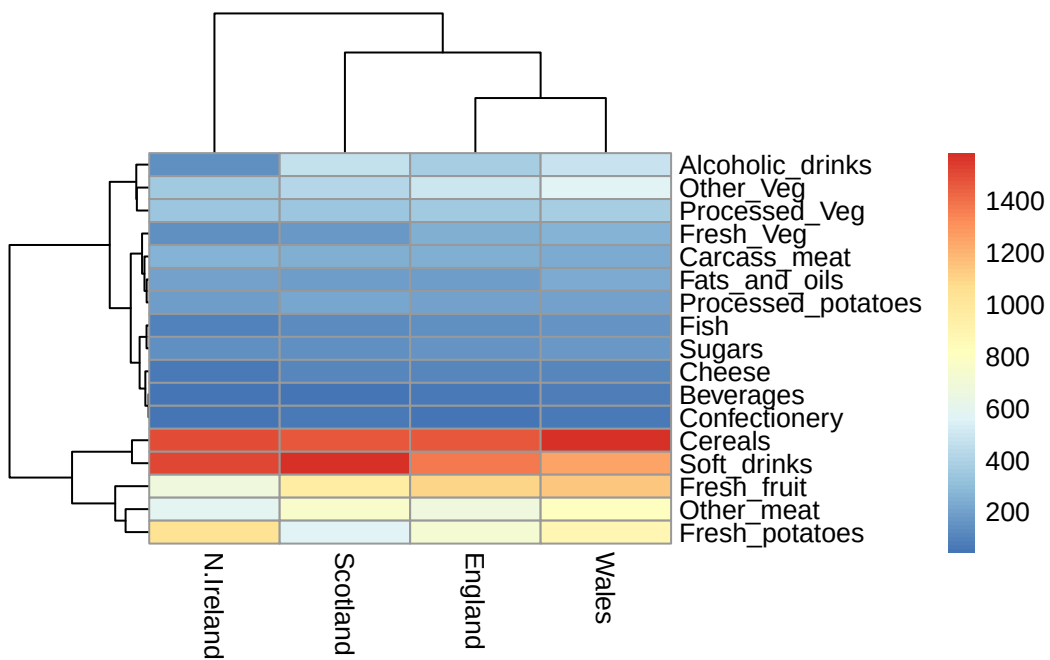


Pairs plots and heatmaps

```
pairs(x, col=rainbow(nrow(x)), pch=16)
```



```
library(pheatmap)
pheatmap( as.matrix(x) )
```



PCA to the rescue

The main PCA function in “base R” is called `prcomp()`. This function wants the transpose of our food data as input (i.e. the foods as columns and the countries as rows).

```
pca <- prcomp(t(x))
```

```
summary(pca)
```

Importance of components:

	PC1	PC2	PC3	PC4
Standard deviation	324.1502	212.7478	73.87622	2.7e-14
Proportion of Variance	0.6744	0.2905	0.03503	0.0e+00
Cumulative Proportion	0.6744	0.9650	1.00000	1.0e+00

```
attributes(pca)
```

\$names

```
[1] "sdev"      "rotation" "center"    "scale"     "x"
```

\$class

```
[1] "prcomp"
```

To make one of our main PCA result figures we turn to `pca$x` the scores along our new PCs. This is called “PC plot” or “score plot” or “ordination plot” ...

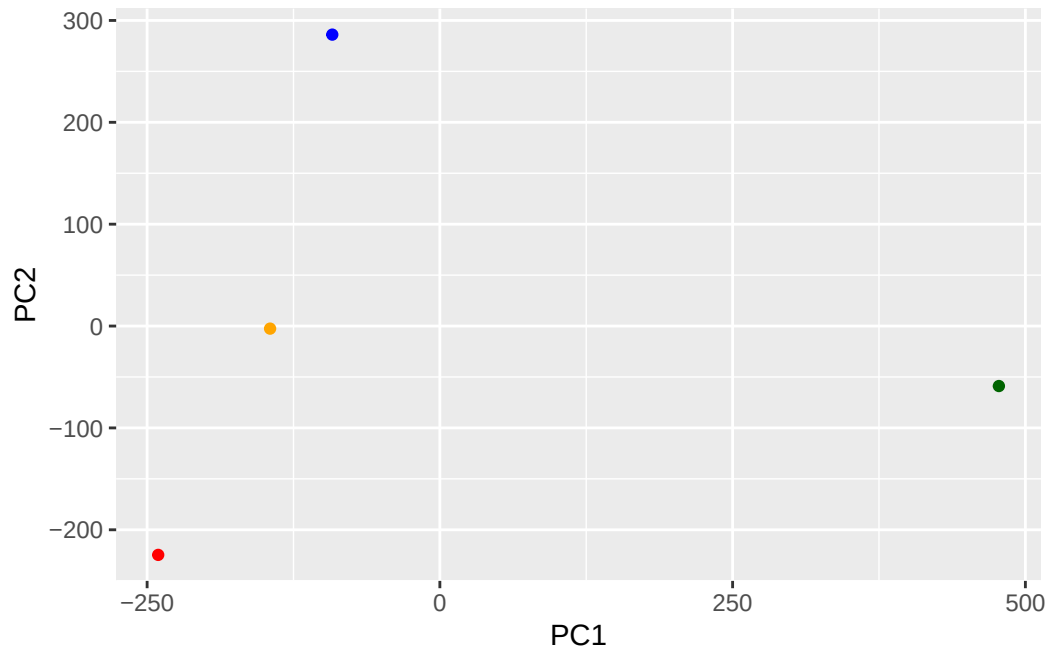
```
pca$x
```

	PC1	PC2	PC3	PC4
England	-144.99315	-2.532999	105.768945	1.612425e-14
Wales	-240.52915	-224.646925	-56.475555	4.751043e-13
Scotland	-91.86934	286.081786	-44.415495	-6.044349e-13
N.Ireland	477.39164	-58.901862	-4.877895	1.145386e-13

```
my_cols <- c("orange", "red", "blue", "darkgreen")
```

```
library(ggplot2)
```

```
ggplot(pca$x)+  
  aes(PC1, PC2)+  
  geom_point(col=my_cols)
```



The second major result figure is called a “loadings plot” of “variable contributions plot” or “weight plot”

```
ggplot(pca$rotation)+  
  aes(PC1, rownames(pca$rotation))+  
  geom_col()
```

